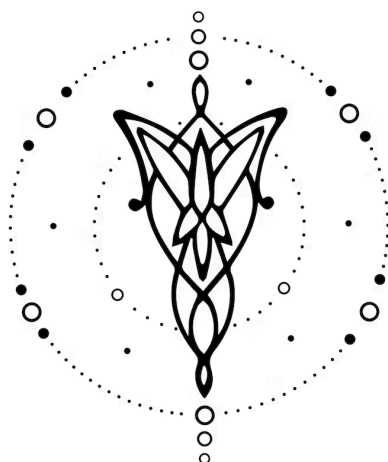




# Compilador de Texto Plano a Presentación

DISEÑO E IMPLEMENTACIÓN

v1.0.0

|                    |   |
|--------------------|---|
| 1. Equipo          | 1 |
| 2. Repositorio     | 1 |
| 3. Dominio         | 2 |
| 4. Construcciones  | 2 |
| 5. Casos de Prueba | 2 |
| 6. Ejemplos        | 3 |

## 1. Equipo

| Nombre   | Apellido          | Legajo | E-mail                                                                           |
|----------|-------------------|--------|----------------------------------------------------------------------------------|
| Sol      | Medina            | 63567  | <a href="mailto:somedina@itba.edu.ar">somedina@itba.edu.ar</a>                   |
| Santiago | Sanchez Marostica | 64056  | <a href="mailto:ssanchezmarostica@itba.edu.ar">ssanchezmarostica@itba.edu.ar</a> |
| Martín   | Gallardo Bertuzzi | 63510  | <a href="mailto:mgallardobertuzzi@itba.edu.ar">mgallardobertuzzi@itba.edu.ar</a> |

## 2. Repositorio

La solución y su documentación serán versionadas en: [TextoPlano-Presentación](#)

## 3. Dominio

El objetivo principal de este proyecto es el desarrollo de un lenguaje de programación especializado (DSL) para facilitar la creación de presentaciones mediante una sintaxis simplificada. Este lenguaje permitirá a los usuarios redactar diapositivas en un formato similar al de **texto plano**, las cuales serán transformadas automáticamente a un código válido de presentación en **LaTeX Beamer** o en **HTML para slides**, listo para compilar y visualizar.

El enfoque principal del DSL es la **manipulación y conversión de estructuras de texto a un formato comprensible por los sistemas de presentación**, como Beamer o reveal.js. El lenguaje debe ser intuitivo y accesible, permitiendo a los usuarios centrarse en el contenido y la organización de sus diapositivas sin preocuparse por los detalles específicos de la implementación del formato final.

Este DSL proporciona a los usuarios herramientas para definir títulos de slides, listas de ítems, imágenes, bloques de código y notas del orador, todo con una sintaxis clara y directa. La idea es que el lenguaje permita la traducción de texto plano a diapositivas listas para presentar, brindando un entorno de trabajo más ágil y productivo.

## 4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones y funcionalidades:

- (I). **Definición de diapositivas** mediante títulos (#).
- (II). **Bullets** o ítems dentro de cada slide usando -.
- (III). Inserción de **imágenes** con @img "path" "caption".
- (IV). Inserción de bloques de **código** con @code ... @end.
- (V). **Notas del orador** con @note "texto".
- (VI). **Subsecciones** opcionales con ##.
- (VII). **Separador de slide** explícito con ---.
- (VIII). Bloques destacados o “callouts” con @block tipo=... “Título” contenido @end
- (IX). **Comentarios** con /\* ... \*/.

## 5. Casos de prueba

### Aceptación

- (I). Una presentación con un único slide y un título.
- (II). Una presentación con dos slides, cada uno con bullets.
- (III). Un slide con una imagen (@img).
- (IV). Un slide con un bloque de código (@code ... @end).

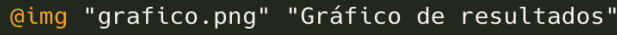
- (V). Un slide con notas para el orador (@note).
- (VI). Un slide que incluya un bloque de definición con @block tipo=alert "Autómatas Finitos" y texto dentro.
- (VII). Un slide que incluya un bloque de ejemplo con @block tipo=example y texto dentro (sin título).

## Rechazo

- (VI). Un slide sin título (falta #).
- (VII). Uso de --- sin que exista un slide definido.
- (VIII). Imagen sin path o sin caption → error.
- (IX). Bloque de código sin cierre @end.
- (X). Uso de componente desconocido (@foo).
- (XI). Título # sin espacio después del símbolo.
- (XII). Bloque @block sin cierre @end.
- (XIII). Bloque @block con tipo desconocido (distinto de normal, alert o example).

## 6. Ejemplos

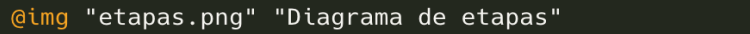
```
# Introducción
- Objetivo del trabajo
- Metodología

# Resultados

@note "Explicar tendencias observadas"
```

Ejemplo 1: Presentación mínima (una slide)

```
# Agenda
- Introducción
- Desarrollo
- Conclusiones

---

# Desarrollo
## Etapa 1
En esta etapa se definió la gramática del lenguaje.
## Etapa 2
Se implementó el parser y los chequeos semánticos.


---

# Preguntas
¿Dudas o comentarios?
@note "Dar espacio para preguntas del público"
```

Ejemplo 2: Tres slides con bulletpoints, título, subtítulo, imagen y notas de orador

```

# Introducción
- Objetivo del proyecto
- Alcance del trabajo
@note "Mencionar antecedentes y motivación"

---

# Marco Teórico
## Conceptos clave
- Autómatas
- Gramáticas
- Parsers
Estos son los fundamentos de la materia.

---

# Resultados
 "Gráfico de resultados obtenidos"
@note "Resaltar que el segundo experimento tuvo mejor desempeño"

---

# Código de Ejemplo
@code
for(i = 0; i < 5; i++) {
    printf("%d\n", i);
}
@end

---

# Conceptos Clave

@block tipo=alert "Definición"
Un lenguaje de programación especializado es aquel diseñado para un
dominio específico.
@end

@block tipo=example "Ejemplo"
SlideLang permite generar presentaciones a partir de texto plano.
@end

---

# Conclusiones
- Logramos implementar el DSL completo
- Se verificaron casos de aceptación y rechazo
- El resultado final es una presentación compilable

```

Ejemplo 3: Presentación con varias slides y distintas etiquetas

Con el ejemplo anterior se obtiene:

## Introducción

### Demo SlideLang

Equipo

Cátedra TLA

25 de agosto de 2025

- Objetivo del proyecto
- Alcance del trabajo

### Marco Teórico

Conceptos clave

- Autómatas
- Gramáticas
- Parsers

Estos son los fundamentos de la materia.

### Resultados

grafico.png

### Código de Ejemplo

```
for(i = 0; i < 5; i++) {  
    printf("%d\n", i);  
}
```

### Conceptos Clave

#### Definición

Un lenguaje de programación especializado es aquel diseñado para un dominio específico.

#### Ejemplo

SlideLang permite generar presentaciones a partir de texto plano.

### Conclusiones

- Logramos implementar el DSL completo
- Se verificaron casos de aceptación y rechazo
- El resultado final es una presentación compilable